Univerzitet u Beogradu
Elektrotehnički fakultet

Jovan Savić i Vuk Dinić, 2022/0206, 2022/0689

Sistem za daljinski nadzor ekrana i zvuka

projekat iz predmeta Principi modernih telekomunikacija

mentor:
Prof. dr Milan Bjelica

Beograd, Oktobar 2025.

Sažetak

U ovom radu prikazan je sistem za daljinski nadzor računara koji omogućava prenos slike ekrana i zvuka u realnom vremenu, kao i osnovnu daljinsku kontrolu ulaznih uređaja. Sistem je zasnovan na arhitekturi tipa klijent–server i sastoji se od dva glavna modula: agenta, koji se pokreće na nadziranom računaru i šalje podatke, i supervisora (servera), koji prima, dekodira i prikazuje video i audio tokove putem grafičkog interfejsa.

Sistem je koncipiran kao centralizovana mrežna arhitektura u kojoj jedan server (supervisor) može istovremeno da prihvati i nadzire veći broj klijenata (agenata).

Ključne reči: supervisor, agent, PCM, digitalizacija signala, kvantizacija, uzorkovanje, audio prenos, mreža, blok podaci, binarni kod, analogni signal

Sadržaj

1 Uvod **4**
1.1 Predmet rada 4
1.2 Motivacija i ciljevi 5

2 Arhitektura sistema **6**

3 Grafički interfejs (GUI) **8**
3.1 Agent GUI 8
3.2 Supervisor GUI 9

4 Prenos podataka **12**
4.1 Video prenos 12
4.2 Audio prenos 13
4.3 Sinhronizacija 15

5 Računica i tehnička analiza **17**
5.1 Audio prenos 17
5.2 Video prenos - nekompresovani RGB 18

6 Performanse **19**

7 Rezultati i diskusija **21**

8 Pokretanje i kompajlovanje projekta **22**
8.1 Struktura direktorijuma 22
8.2 Pokretanje aplikacija 23
8.3 Ručno kompajlovanje (PyInstaller) 24
8.4 Napomena o kompatibilnosti 24

9 Literatura **25**

Spisak slika

2.1 Arhitektura: supervisor i agenti povezani trokanalnom komunikacijom 7

4.1 Princip PCM: sinusoidalni signal, uzorci i nekoliko kvantizacionih nivoa za ilustraciju 14

Spisak tabela

6.1 Performanse sistema sa 5 agenata 19

Glava 1

Uvod

1.1 Predmet rada

Integrisani sistem za real-time streaming multimedijalnih podataka

- Prenos video i audio signala u realnom vremenu
- Sinhronizacija medijskih tokova
- Tehnička i praktična analiza performansi

Tehnička analiza i implementacija sistema za sinhronizovani multimedijalni prenos

- Teorijsko modelovanje prenosa podataka
- Implementacija video i audio stream-a
- Evaluacija sinhronizacije i kvaliteta signala

Optimizovani sistem za streaming audio-video sadržaja u realnom vremenu

- Efikasna kompresija i prenos podataka
- Sinhronizacija audio i video tokova
- Analiza tehničkih performansi i rezultata

1.2 Motivacija i ciljevi

Svakodnevna upotreba i velika potražnja za striming servisima u edukativne i zabavne svrhe motiviše razvoj pouzdanih sistema za prenos video i audio podataka preko mreže. Proučavanje i implementacija takvih sistema omogućava bolje razumevanje tehnologija iza modernih striming rešenja i poboljšava kvalitet prenosa multimedije.

Ciljevi rada:

- Analizirati postojeće metode i protokole za prenos multimedijalnih podataka u realnom vremenu
- Implementirati sistem za prenos video i audio signala sa sinhronizacijom medijskih tokova
- Izvršiti tehničku evaluaciju performansi sistema kroz teorijsku analizu i praktična merenja
- Predložiti optimizacije za poboljšanje kvaliteta prenosa i smanjenje latencije

Glava 2

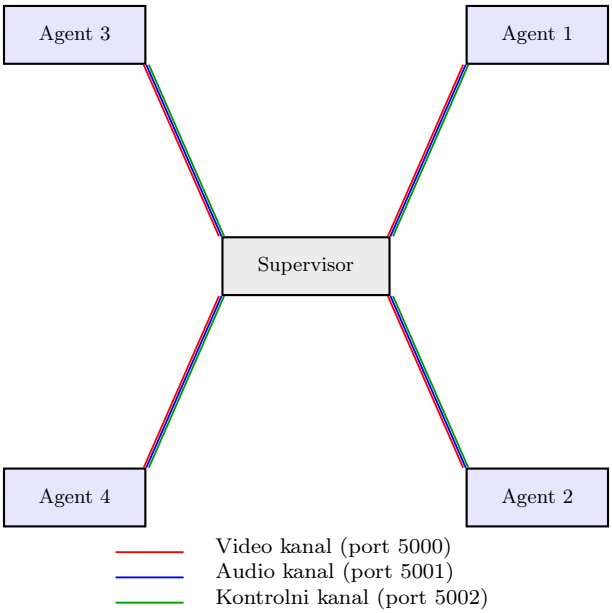
Arhitektura sistema

Sistem je organizovan u centralnu strukturu u kojoj **supervisor** ima ulogu glavnog kontrolnog čvora. On prima video i audio podatke od svih priključenih **agenata** (klijenata) u mreži. Svaki agent po priključenju automatski započinje slanje podataka sa svog kraja, čime se omogućava dinamično proširenje mreže bez potrebe za ručnim konfigurisanjem.

Svaki čvor u sistemu, bilo da je u pitanju supervisor ili agent, koristi tri odvojena komunikaciona kanala, odnosno tri mrežna porta:

- **Video port (5000)** – koristi se za prenos video okvira. Podaci se šalju u komprimovanom obliku kako bi se smanjila zauzetost mreže i kašnjenje.
- **Audio port (5001)** – namenjen je prenosu audio signala, sinhronizovanog sa video tokom. Prenos se odvija paralelno sa video kanalom.
- **Kontrolni port (5002)** – služi za razmenu kontrolnih poruka između supervisor-a i agenata, kao što su potvrde o prijemu podataka, sinhronizacija, upravljanje konekcijama i komande za početak ili prekid prenosa.

Takva organizacija omogućava da se svaki tip podataka obrađuje nezavisno, čime se povećava stabilnost sistema i pojednostavljuje kontrola mrežnih tokova.



Slika 2.1: Arhitektura: supervisor i agenti povezani trokanalnom komunikacijom

Glava 3

Grafički interfejs (GUI)

Grafički interfejs (GUI) predstavlja centralni vizuelni sloj aplikacije koji omogućava nadgledanje i upravljanje radom sistema. Implementiran je u okviru modula `supervisor i agent` korišćenjem `Tkinter` biblioteke u Pythonu. GUI povezuje korisnika sa mrežnim i procesnim delom sistema, pružajući pregled aktivnih agenata, video prenosa i kontrolnih funkcija. Preko grafickog okruzenja je realizovan i unos odredjenih parametra koji se koriste u radu programa.U nastavku je dataljno opisan graficki interfejs obe strrane.

3.1 Agent GUI

Grafički interfejs agenta je jednostavan i funkcionalan. Njegova osnovna uloga je da omogući korisniku unos identifikacionih podataka i IP adrese supervisora.

Početni prozor i unos parametara Nakon pokretanja, otvara se glavni prozor pod nazivom **„Agent – Surveillance“**. Prozor sadrži dva polja za unos:

- **Agent ID** – jedinstveni identifikator agenta (npr. `agent001`), koji se koristi za identifikaciju prilikom uspostavljanja veze sa supervisorom.Po uspesnoj uspostavljenoj konekciji sa supervisorom, korisnicki ID se pojavljuje u tabeli kod supervisor-a
- **Supervisor IP** – IP adresa računara na kojem se pokreće supervisor. Ova adresa se koristi pri povezivanju na tri mrežna porta (5000, 5001, 5002).

Ispod ovih polja nalazi se dugme **Connect**, kojim se započinje proces povezivanja.Pri uspesnom povezivanju zatvara se sam GUI agenta i konekcija nastavlja sa "radom".Sve greske se ispisuju u konzoli, ako je aplikacija pokrenuta preko izvrsnog fajla korisnik nece moci da ih vidi.

3.2 Supervisor GUI

Supervisor GUI je centralni kontrolni panel koji omogućava nadgledanje i upravljanje svim priključenim agentima. Predlaže se da se ovaj proces prvi pokrene i unesu podaci kako ne bi agent pokušavao da se poveže sa supervizorom koji još uvek nije podignut.

1. Login prozor (LoginFrame) Prvi prozor koji se pojavljuje pri pokretanju Supervisor aplikacije je `LoginFrame`. Njegove ključne funkcije:

- **Unos korisničkih podataka** — tipičan login (npr. `admin/admin`) za pristup glavnom interfejsu.
- **IP bind** — polje u kojem se unosi IP adresa na kojoj će Supervisor da „binduje" svoje server sokete (npr. `0.0.0.0` ili određena lokalna IP adresa). Ta IP adresa određuje na kojoj interfejsi Supervisor osluškuje dolazne konekcije.
- Po uspešnom logovanju otvara se `MainFrame` (glavni prozor).

2. Main prozor — pregled agenata (Tabela) i kontrole Glavni prozor ima više zona: lista/tabela agenata, grid sa thumbnailima, detaljni prikaz selektovanog agenta, kontrolne komande,dugme za preuzimanje kontrole i status/log panel.

Lista agenata (tabela)

- U tabeli su prikazani svi poznati/registrovani agenti (Agent ID, IP adresa/port, status konekcije, vreme poslednje poruke).
- Kada agent uspostavi vezu, Supervisor u pozadinskoj niti prima njegov 64-bajtni `agent_id` i upisuje ga u internu mapu `agents`. Unos se pojavljuje u tabeli tek nakon osveženja tabele (videti dugme `Refresh`).
- **Dugme** `Refresh` se nalazi ispod tabele i mora ga korisnik pritisnuti kako bi tabela osvežila prikaz i prikazala novokonektovane agente. (U trenutnoj implementaciji tabela ne osvežava se automatski na svaki novi dolazak; dugme je namerno dodato da korisnik kontrolše trenutak osveženja.)
- Ako agent bude prekinut, njegov status je označen (npr. `disconnected`) i on se može ukloniti iz tabele pozivom funkcije za osveženje(pritiskom Refresh dugmeta) koja briše neaktivne unose.

Grid pregled (thumbnail grid)

- U centralnom delu nalazi se grid koji prikazuje male sličice (thumbnail) poslednjih primljenih frejmova od aktivnih agenata.
- Grid je statičan u smislu rasporeda (fiksan broj kolona/redova) — korisnik može videti više agenata odjednom. Dodatno osveženje grid prikaza se vrši periodičnim osvežavanjem u zasebnoj niti.
- Kada agent izgubi konekciju, njegov thumbnail se uklanja pri sledećem osveženju grida, tako da grid ne prikazuje neaktivne agente.

Detajlni (pojedinačni) prikaz agenta

- Selektovanjem reda u tabeli ili dvostrukim klikom na thumbnail otvara se (kreira novi) poseban prozor ili panel sa detaljnim prikazom izabranog agenta.
- Taj prozor sadrži:
  - veći video prikaz ,
  - kontrole za snimanje screenshota (dugme `Save screenshot`),
  - prekidač za reprodukciju/zaustavljanje audio toka (ako je audio aktivan),
  - dugme `Take control` / `Preuzmi kontrolu` koje šalje odgovarajuću JSON komandu preko kontrolnog kanala (port `5002`) agentu da bi ovaj emulirao događaje miša/tastature supervisora,
  - indikatore (latencija, poslednji timestamp, bitrate).
- U opisanoj implementaciji važno je naglasiti da se **tabela zahteva ručno osvežava** da bi se novi agent video u njoj.

Kontrolni panel i operacije nad agentima

- Pored tabele, nalaze se dugmad za:
  - **Refresh** — osvežava tabelu i grid, pokazuje nove agente i uklanja neaktivne;
  - **Disconnect selected** — prekida konekciju izabranog agenta (zatvara odgovarajuće sokete na server strani i uklanja agenta iz interne strukture);
  - **Start/Stop stream** — komande za start/stop slanja (preko kontrolnog kanala).
- Operacije nad agentima (npr. `Take control`) se realizuju slanjem JSON poruka sa headerom dužine (4 bajta, big-endian) koje agent razume i izvršava.

3. Status i log Supervisor GUI prikazuje real-time statuse:

- stanje slušanih portova (vezan IP i portovi 5000/5001/5002),
- listu aktivnih soketa i broj povezanih agenata,
- log događaja (konekcije, diskonekcije, greške pri dekodiranju frejmova, pokušaji preuzimanja kontrole).

4. Veza između Supervisor i Agenta (kratko objašnjenje protokola)

- Agent pri povezivanju prema Supervisoru uspostavlja tri TCP veze (video, audio, kontrola) i odmah šalje fiksni 64-bajtni `agent_id` da bi se identifikovao.
- Supervisor čitajući taj identifikator zna kojem agentu pripadaju budući paketi sa te konekcije (i povezuje soket sa mapom `agents[agent_id]`).
- Za video i kontrolne poruke koristi se prefiks od 4 bajta koji sadrži dužinu payload-a (format `struct.pack('>I', len(data))`), tako da prijemna strana zna koliko bajtova treba da pročita.

5. Napomene o ponašanju GUI-ja u tipičnim scenarijima

- **Povezivanje novog agenta:** agent se poveže na Supervisor (server bindovan na uneti IP). Supervisor prima agent_id i zapisuje ga, ali korisnik mora pritisnuti dugme `Refresh` da bi tabela i grid prikazali novog agenta. Ovo omogućava kontrolisano osvežavanje i izbegava vizuelni „skok" u slučaju masovnih konekcija.
- **Prekid povezivanja:** nakon što agent prekine vezu, njegov unos ostaje u internim strukturama dok se korisnik ne odluči da osveži prikaz; pri osveženju neaktivni unosi se uklanjaju.
- **Preuzimanje kontrole:** kada Supervisor pošalje komandu za preuzimanje kontrole, agent će je primiti preko `ControlListener`-a i izvršiti događaje koristeći `pynput`. GUI supervisora treba da prikaže status te komande i eventualne greške.

Glava 4

Prenos podataka

4.1 Video prenos

Video prenos se odvija tako što agent periodično hvata sadržaj ekrana i šalje ga **supervisoru** preko porta **5000**. Svaki video okvir (frame) se kompresuje korišćenjem `OpenCV` i `numpy` biblioteka, čime se značajno smanjuje količina podataka koji se šalju mrežom.

Prenos se realizuje u sledećim koracima:

1. Agent hvata trenutni prikaz ekrana pomoću biblioteke `mss`.
2. Dobijena slika se konvertuje u `JPEG` format i kodira u niz bajtova.
3. Na početak svakog paketa dodaje se **header** koji sadrži informaciju o veličini slike u bajtovima.
4. Supervisor, koji stalno osluškuje port 5000, najpre čita header da bi znao koliko bajtova pripada trenutnom okviru, a zatim preuzima ceo sadržaj slike i dekodira je.

Header je realizovan kao binarni zapis veličine slike u fiksnoj dužini od **4 bajta**, uz korišćenje Python-ovog modula `struct` i formata `!I`, koji omogućava dosledno kodiranje integer vrednosti nezavisno od platforme.

Primer:

```
header = struct.pack("!I", len(frame_data))
```

Korišćenje headera omogućava da se tačno zna gde jedan video okvir završava, što je ključno za ispravno dekodiranje slike na strani prijemnika i sprečava gubitak sinhronizacije između paketa.

Tipična veličina prenete slike iznosi između **30 kB i 150 kB**, u zavisnosti od rezolucije (npr. 640x480 piksela) i nivoa kompresije. Ovakav pristup omogućava stabilan i kontinuiran video prenos sa malim kašnjenjem, čak i u uslovima ograničenog protoka mreže.

4.2 Audio prenos

Audio prenos se odvija tako što agent hvata zvuk sa svog mikrofona i šalje ga **supervisoru** preko porta **5001**. Zvuk se obrađuje u malim blokovima (frames) i kompresuje se kako bi zauzeo manje mrežne propusnosti i omogućio real-time prenos.

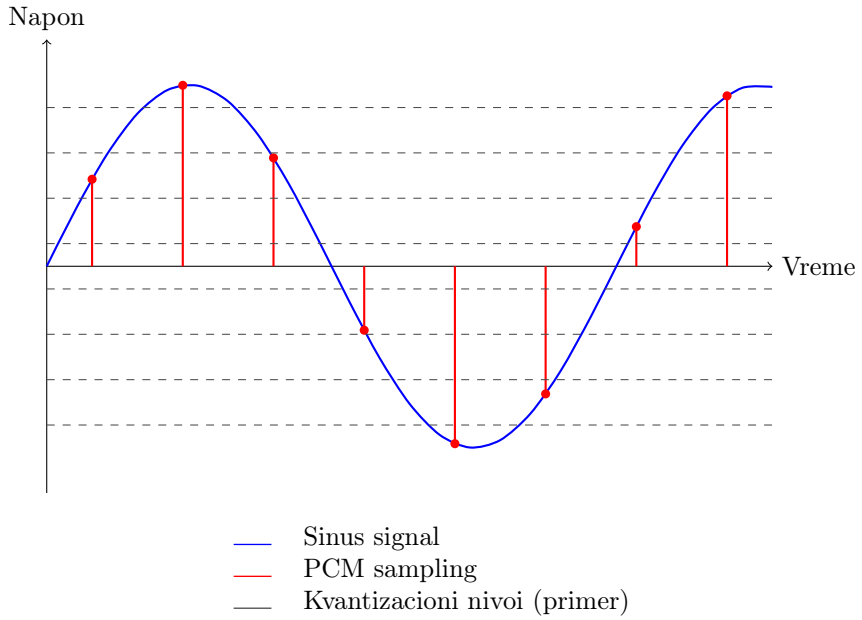
Prenos se realizuje u sledećim koracima:

1. Agent prikuplja audio signal pomoću biblioteke `pyaudio`, u blokovima od npr. 1024 uzorka.
2. Svaki blok audio podataka se kodira u binarni niz bajtova PCM tehnikom.
3. Na početak svakog bloka dodaje se **header** koji sadrži dužinu bloka u bajtovima.
4. Supervisor, koji osluškuje port 5001, najpre čita header da bi znao koliko bajtova pripada trenutnom audio bloku, a zatim preuzima podatke i dekodira ih za reprodukciju ili dalje procesiranje.

Header je realizovan kao fiksna 4-bajtna vrednost, koristeći Python modul `struct` i format `!I`, što omogućava dosledno kodiranje veličine bloka nezavisno od platforme:

```
header = struct.pack("!I", len(audio_data))
```

Korišćenje headera omogućava supervisoru da pravilno prepozna granice audio blokova i sinhronizuje reprodukciju sa video tokom. Tipična veličina audio bloka zavisi od uzorka po sekundi (npr. 44100 Hz) i dužine bloka (1024 ili 2048 uzoraka), a prenosi se kontinuirano kako bi se obezbedio real-time prenos zvuka sa minimalnom latencijom.



Slika 4.1: Princip PCM: sinusoidalni signal, uzorci i nekoliko kvantizacionih nivoa za ilustraciju

Objašnjenje PCM-a

Pulse Code Modulation (PCM) funkcioniše na sledeći način:

- Originalni analogni signal se u određenim vremenskim intervalima uzorkuje.
- Svaka uzeta vrednost u periodi se **zaokružuje na najbliži kvantizacioni nivo**.
- Nakon zaokruživanja, vrednost se **binarno kodira** u fiksni broj bita koji predstavlja nivo.
- Na taj način se analogni signal pretvara u digitalni tok podataka koji se može prenositi i obrađivati.

4.3 Sinhronizacija

U sistemu je implementirana sinhronizacija između audio, video i kontrolnih tokova koristeći *odvojene niti* za svaki kanal u okviru svakog čvora (supervisor i agent).

Pristup zajedničkim resursima (poput bafera i socket-a) je *zaštićen mehanizmima međusobnog isključivanja (mutex/lock)* kako bi se izbegli konflikti i gubici podataka.

Za sinhronizaciju i obradu podataka korišćeni su standardni algoritmi za *redove poruka (queue), blokiranje niti (thread blocking) i semafore (semaphores)*, što omogućava precizan i stabilan prenos podataka.

Ova arhitektura omogućava:

- Paralelni prenos audio i video podataka bez međusobnog zastoja.
- Održavanje real-time performansi uz minimalno kašnjenje.
- Jednostavno skaliranje sistema sa više agenata.

Napredna sinhronizacija tokova:

Pored osnovne međunitične sinhronizacije, sistem implementira i *vremensku sinhronizaciju* između agenata i supervisora. Svaki poslati blok (bilo audio, video ili kontrolni) nosi **vremensku oznaku (timestamp)** koja označava trenutak kada je podatak generisan na strani agenta. Supervisor koristi ove oznake kako bi poravnao sve pristigle tokove u vremenu i obezbedio da se audio i video reprodukuju sinhronizovano.

U praktičnoj implementaciji, zbog varijabilnosti mrežnog kašnjenja (*latency*) i fluktuacija u brzini prijema paketa (*jitter*), sistem koristi male međubaferne memorije (`queue.Buffer`) koje omogućavaju:

- Privremeno skladištenje nekoliko frejmova unapred radi kompenzacije kašnjenja.
- Poravnavanje video i audio okvira prema njihovim vremenskim oznakama.
- Smanjenje vizuelno-čujne razlike između slike i zvuka u realnom vremenu.

Mehanizam za korekciju kašnjenja:
Supervisor periodično meri prosečno vreme prijema paketa po agentu i prilagođava veličinu bafera dinamički. Ako detektuje povećano kašnjenje u video toku, privremeno zadržava audio blokove dok ne pristignu odgovarajući video frejmovi. Ova adaptivna kontrola omogućava stabilniji prikaz, naročito pri radu sa više agenata u istoj mreži.

Detekcija i obrada gubitka paketa:
U slučaju prekida konekcije ili gubitka paketa, supervisor identifikuje prekid u nizu vremenskih oznaka. Tada:

- automatski popunjava praznine dupliciranjem poslednjeg primljenog frejma (tzv. *frame hold*),

- nastavlja reprodukciju bez prekida prikaza,
- beleži događaj u log fajl za kasniju analizu.

Ova strategija omogućava sistemu da održi kontinuitet prikaza čak i pri kratkotrajnim prekidima veze ili povećanom mrežnom opterećenju.

Paralelizam i prioriteti niti:
Za svaku vrstu toka (video, audio, kontrolni) kreirana je posebna nit sa prilagođenim prioritetom izvršavanja:

- *Video nit* ima srednji prioritet – obezbeđuje konzistentan broj frejmova po sekundi.
- *Audio nit* ima viši prioritet – jer kašnjenje u zvuku korisnik lakše primeti od vizuelnog.
- *Kontrolna nit* ima niži prioritet – obrađuje manje kritične komande i potvrde.

Zahvaljujući ovom modelu raspodele prioriteta, sistem uspešno održava balans između brzine, kvaliteta i stabilnosti, čak i pri većem broju aktivnih agenata. Na ovaj način se ostvaruje real-time sinhronizacija multimedijalnih tokova bez značajnog povećanja procesorskog opterećenja.

Zaključak sinhronizacije:
Uprkos složenosti mrežnog prenosa, primenjeni pristup omogućava postizanje kašnjenja manjeg od 100 ms između video i audio toka, što zadovoljava kriterijume za real-time nadzor i komunikacione aplikacije.

Glava 5

Računica i tehnička analiza

Za početne i referentne vrednosti korišćene u sistemu uzimane su što bliže industrijskim standardima kako bi performanse bile relevantne i praktične.

5.1 Audio prenos

Trenutni parametri audio prenosa su:

- RATE = 22050 Hz
- CHANNELS = 1
- CHUNK = 2048
- Format = 16-bit PCM

Bitrate po kanalu se računa kao:

$$\text{Bitrate} = \text{RATE} \times \text{CHANNELS} \times \text{Bajt po uzorku} \times 8$$

$$\text{Bitrate} = 22050 \times 1 \times 2 \times 8 = 352800 \text{ bit/s} \approx 343 \text{ kb/s}$$

Trajanje jednog audio bloka (CHUNK):

$$t_{\text{chunk}} = \frac{\text{CHUNK}}{\text{RATE}} = \frac{2048}{22050} \approx 0.093 \text{ s}$$

Dakle, sistem šalje oko 10 audio blokova u sekundi.

Proširenje na 44.1 kHz i stereo

Za stereo audio i standardnu frekvenciju od 44.1 kHz:

$$\text{Bitrate} = 44100 \times 2 \times 2 \times 8 = 1\,411\,200 \text{ bit/s} \approx 1.34 \text{ Mb/s}$$

Povećanje je linearno sa brojem kanala. Sistem se može proširiti tako da:

- svaki audio kanal koristi zaseban TCP socket
- prenos se odvija paralelno i sinhronizovano
- omogućava stereo i višekanalni (surround) prenos bez gubitka kvaliteta

Takvo rešenje može dostići standard **44.1 kHz stereo** kvalitet zvuka uz minimalno povećanje latencije.

5.2 Video prenos - nekompresovani RGB

Trenutni parametri video prenosa u projektu:

- Rezolucija: 640×480 piksela
- Broj frejmova: 30 fps
- Boje po pikselu: 24 bita (RGB)
- Kompresija: trenutno nije korišćena (nekompresovan prenos)

Bitrate nekompresovanog video signala

Bitrate se računa kao:

$$\text{Bitrate} = \text{Width} \times \text{Height} \times \text{Bojanje po pikselu} \times \text{FPS}$$

Za naše parametre:

$$\text{Bitrate} = 640 \times 480 \times 24 \times 30 = 221\,184\,000 \text{ bit/s} \approx 221 \text{ Mbps}$$

Trajanje jednog frejma

$$t_{\text{frame}} = \frac{1}{\text{FPS}} = \frac{1}{30} \approx 0.033 \text{ s}$$

Dakle, sistem šalje oko 30 frejmova u sekundi.

Ukupna propusnost

$$\text{Ukupna propusnost video + audio} \approx 221 \text{ Mbps} + 0.34 \text{ Mbps} \approx 221.34 \text{ Mbps}$$

Glava 6

Performanse

Sva merenja su vršena na računarima u prostorijama fakulteta, koristeći lokalnu mrežu sa jednim supervisorom i pet agenata. Tokom testiranja izmereni su sledeći pokazatelji performansi:

Tabela 6.1: Performanse sistema sa 5 agenata

Parametar	Vrednost
Latency (video)	30-50 ms
Latency (audio)	90-100 ms
Propusnost (video)	221 Mbps
Propusnost (audio)	343 kbps
CPU utilizacija (supervisor)	25-35%
CPU utilizacija (agent)	15-25%
Memorijsko opterećenje (supervisor)	150-200 MB
Memorijsko opterećenje (agent)	50-80 MB

Način reprodukcije rezultata:

- **Latencija video signala:** Može se meriti pokretanjem stoperice na računaru supervisora i istovremenim beleženjem ekrana agenta preko screen share-a. Razlika između prikazanog trenutka i stvarnog prenosa omogućava određivanje kašnjenja slike.
- **Latencija audio signala:** Precizno se može odrediti tako što se audio signal sa agenta i sa supervisora snimi, a zatim u programu poput *Audacity* uporede karakteristični pikovi zvuka. Razlika u vremenu između tih pikova predstavlja latenciju audio prenosa.
- **CPU opterećenje:** Očitava se direktno iz Task Manager-a tokom normalnog rada sistema, prateći kako procesor reaguje na paralelni prenos podataka od svih pet agenata.

- **Memorijsko opterećenje:** Procena količine memorije zauzete za baferovanje audio i video podataka, što pomaže u proveri da li je sistem sposoban da održava real-time prenos.
- **Propusnost:** Izračunava se prema veličini paketa i broju frejmova/audio blokova poslatih u sekundi, omogućavajući kontrolu ukupnog protoka podataka kroz mrežu.

Glava 7

Rezultati i diskusija

Analiza eksperimentalnih rezultata ukazuje da sistem funkcioniše stabilno u realnom vremenu i da je arhitektura sa odvojenim nitima za audio, video i kontrolne tokove efikasna. Zabeležena latencija i opterećenje resursa pružaju osnovu za identifikaciju potencijalnih optimizacija.

Jedno od ključnih poboljšanja je uvođenje video kompresije (npr. H.264), što bi omogućilo smanjenje propusnosti, povećanje broja frejmova ili dodavanje novih agenata bez značajnog povećanja mrežnog opterećenja. Time bi se poboljšala skalabilnost sistema i smanjila upotreba CPU resursa.

Takođe, prelazak sa TCP na UDP protokol za prenos audio i video podataka mogao bi dodatno smanjiti latenciju, jer UDP ne koristi potvrde i retransmisije. To bi omogućilo brži prenos u realnom vremenu, ali bi zahtevalo dodatne mehanizme za korekciju gubitaka paketa i očuvanje sinhronizacije tokova.

Dodatno, rezultati ukazuju na mogućnost poboljšanja sinhronizacije i balansiranja resursa između više agenata, što bi doprinelo stabilnijem radu u mrežama sa većim brojem korisnika. Ova analiza pruža jasne smernice za buduća unapređenja sistema u cilju veće efikasnosti i skalabilnosti.

Glava 8

Pokretanje i kompajlovanje projekta

Projektni direktorijum sadrži sve neophodne komponente za rad aplikacije, organizovane tako da omogućavaju jednostavno pokretanje, ali i ručno kompajlovanje izvršnih datoteka ukoliko korisnik to želi.

8.1 Struktura direktorijuma

Na vrhu (root) se nalaze sledeće stavke:

- **agent.py** – glavni Python fajl koji pokreće Agent GUI aplikaciju.
- **supervisor.py** – glavni Python fajl koji pokreće Supervisor GUI aplikaciju.
- **agentbuild/** – direktorijum u kojem se nalazi konfiguracija i rezultati kompajlovanja Agenta.
- **supervisor/** – direktorijum koji sadrži izvorne fajlove Supervisora (GUI, logiku servera i pomoćne funkcije).
- **supervisorbuild/** – direktorijum u kojem se nalazi konfiguracija i rezultati kompajlovanja Supervisora.

Unutar direktorijuma **agentbuild/** nalaze se sledeće podstrukture:

- **build/** – privremeni fajlovi koje PyInstaller generiše tokom procesa kompajlovanja.
- **dist/** – sadrži konačan izvršni fajl `peeker.exe`, koji predstavlja gotovu verziju Agenta.

- **peeker.spec** – specifikacija koju PyInstaller koristi za kreiranje izvršnog fajla Agenta.

Analogno tome, direktorijum **supervisorbuild/** sadrži:

- **build/** – privremene datoteke generisane pri izgradnji Supervisora.
- **dist/** – sadrži izvršni fajl `BigBrother.exe`, koji predstavlja gotovu verziju Supervisora.
- **BigBrother.spec** – konfiguracioni fajl za PyInstaller koji definiše način kompajlovanja.

8.2 Pokretanje aplikacija

Projekat se može pokrenuti na dva načina: direktno iz Python koda ili korišćenjem već pripremljenih izvršnih fajlova(peeker.exe i BigBrother.exe).

1. Pokretanje iz Python izvornog koda

Za ovaj način rada potrebno je da korisnik ima instaliran Python (verzija 3.10 ili novija) i sledeće pakete:

```
tkinter, opencv-python, numpy, mss, pyaudio, pynput
```

Pokretanje se vrši komandom:

```
python agent.py
```

za pokretanje Agenta, odnosno:

```
python supervisor.py
```

za pokretanje Supervisora.

2. Pokretanje gotovih izvršnih fajlova (.exe)

Ako se koriste već kompajlovane verzije, potrebno je samo pokrenuti odgovarajući fajl:

- `agentbuild/dist/peeker.exe` – pokreće Agent GUI aplikaciju.
- `supervisorbuild/dist/BigBrother.exe` – pokreće Supervisor GUI aplikaciju.

Ove verzije ne zahtevaju instaliran Python, jer sadrže sve potrebne biblioteke u sebi.

8.3 Ručno kompajlovanje (PyInstaller)

Ukoliko korisnik želi da sam napravi izvršne fajlove, može koristiti **PyInstaller**. Pre pokretanja komande potrebno je instalirati ga:

```
pip install pyinstaller
```

Zatim, iz root direktorijuma, pokretanje komandi izgleda ovako:

```
pyinstaller --onefile --noconsole --icon=icon.png agent.py
pyinstaller --onefile --noconsole --icon=icon.png supervisor.py
```

Ove komande kreiraju izvršne fajlove u odgovarajućim `dist/` folderima. Alternativno, mogu se koristiti već postojeći `.spec` fajlovi:

```
pyinstaller agentbuild/peeker.spec
pyinstaller supervisorbuild/BigBrother.spec
```

8.4 Napomena o kompatibilnosti

Izvršne verzije su kreirane i testirane na operativnom sistemu Windows 11, ali se kod može bez izmene pokrenuti i na Linux distribucijama (npr. Debian ili Ubuntu) direktno iz Python fajlova, jer se koriste kompatibilne biblioteke.

Glava 9

Literatura

[1] MSS – Python Screen Capture Library. Dostupno online: https://pypi.org/project/mss/

[2] OpenCV – Open Source Computer Vision Library. Dostupno online: https://opencv.org/

[3] Tkinter – Python Interface to Tcl/Tk. Dostupno online: https://docs.python.org/3/library/tk.html

[4] Wikipedia contributors, Pulse-code modulation, Wikipedia, The Free Encyclopedia, https://en.wikipedia.org/wiki/Pulse-code_modulation

[5] Python Software Foundation, threading — Thread-based parallelism, Python Standard Library Documentation, dostupno online: https://docs.python.org/3/library/threading.html

[6] The Little Book of Semaphores Allen B. Downey,dostupno online:https://greenteapress.com/wp/semaphores/